VRS41 Monochromator Controller — Manual

Control software for a grating monochromator driven by a FAULHABER brushless servo (`CS-BX4`, integrated motor + controller, RS232) with an NI-DAQ analog input reading the PMT.

Table of contents

---

1. Quick start

```
python3 -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python3 main.py
```

A virtual environment is recommended but not required — the application only needs PyQt6, pyqtgraph and pyserial, plus `nidaqmx` if real hardware is attached.

A normal measurement session:

1. **Connect** — enter the serial port, press *Connect*.
2. **Reference Run** — seats the mechanical backlash so the software knows which way the slack is taken up.
3. **Confirm the wavelength** — a dialog asks what the monochromator's own mechanical counter reads. This anchors the software position to reality.

4. **Go To** the region of interest, check the signal on the live AI readout.
5. Set **Start / End / Step**, set **Timebase** and averaging, press **Start Scan**.
6. The scan is plotted live and (if autosave is on) written to CSV.

> **The single most important thing to understand:** this instrument has **no absolute position sensor**. The drive counts encoder steps, nothing more. Every wavelength the software shows is a *calculated* value, derived from a starting point you told it about. If the grating is moved by hand, or the app is restarted after someone else moved it, the displayed wavelength is wrong until you re-anchor it. Sections 4 and 5 exist entirely because of this.

---

2. The window at a glance

Header — Current λ, State (FSM), Cursor λ / Cursor V (follows the mouse over the plot), and the red **STOP** button.

Left — two tabs:

Tab	Contents
Scope	The live plot, with a *Clear Plot* button on its toolbar
Console	The full log

Right — collapsible sections, in this order:

Section	What lives there
CONNECTION	Port, baud, Connect/Disconnect, status
MOVE	Current λ, Go To, Jog ±, Mark Point
SCAN	Start/End/Step, Timebase, Pre-settle, Start Scan, Free Run, Pause/Resume/Stop
AVERAGING	Avg mode, Avg samples, Avg fraction, live explanation
SCAN QUEUE	Queued scans, Add/Update/Remove/Clear, Run queue
BACKLASH / SLIP	Slip value, Reference Run, calibration dialog
CALIBRATION	Known λ here → re-anchor
DAQ	Device and analog input channel
EXPORT	Autosave, folder, filename pattern, Save CSV now

Footer (status bar) — State · connection · Slip · drive Temp · live AI value with bar graph, max-hold and *Reset max*.

---

3. Connecting

Enter the serial **Port** (`COM3` on Windows, `/dev/tty.usbserial-…` on macOS/Linux) and **Baud** (9600 on this rig). Press **Connect**.

The button is a toggle: teal *Connect* when disconnected, red *Disconnect* when connected.

On connect the software:

1. drains any stale bytes in the serial buffer and logs what it found,

2. sends `EN` (enable), `V0` (velocity mode, 0 rpm), the answer mode and the ramp parameters,
3. reads the position,
4. asks you to confirm the current wavelength (see §4).

Enter **`SIM`** as the port to run against the built-in simulator with no hardware attached (§18).

---

4. Knowing where you are: Reference Run and Calibration

These two are different things and are often confused.

Reference Run (BACKLASH / SLIP section)

Moves a fixed distance in one direction to **seat the mechanical backlash**. It does *not* tell the software where it is — it establishes *which direction the slack is currently taken up in*, so the first real move can be compensated correctly.

Run it once per session, after connecting. It is deliberately **not** persisted between sessions: it reflects the physical state of the gear train, which a closed program cannot know.

```
[REF] Reference run: +3.000 nm (-1085295 steps) to seat backlash.
[REF] Reference run complete. Direction seated at +1. Ready.
```

Calibration — "Known λ here" (CALIBRATION section)

Re-anchors the software's idea of the current wavelength. Type the wavelength the monochromator's own mechanical readout shows, or the wavelength of a line you have peaked on, and press the button.

The motor does **not** move. Only the offset changes; `STEPS_PER_NM` is untouched.

```
[CAL] Current position re-anchored: 590.790 -> 590.690 nm (offset
calibration; motor NOT moved, STEPS_PER_NM unchanged).
```

Both a comma and a dot work as the decimal separator here.

> If you calibrate on one line and a *second* known line is then also off, that is a **scale** error (`STEPS_PER_NM`), not an offset error, and re-anchoring will not fix it.

---

5. Backlash / slip compensation

Gear trains have slack. When the direction of travel reverses, the motor turns for a while before the grating starts to follow. The software compensates by adding extra steps on every reversal:

```
[SLIP] Reversal +1->-1: adding 0.082 nm (+29665 steps) backlash compensation.
```

Those steps **are** counted by the encoder but are **not** wavelength travel — they take up slack. The software accounts for this separately, so a sweep still lands on its nominal end point.

Measuring the value: the calibration dialog

BACKLASH / SLIP → the calibration dialog

With a reference lamp on, the line is crossed **once upward and once downward**. The offset between the two measured peaks is the backlash.

Field	Meaning
Reference line λ (nm)	The known line. Default **587.5618 nm (He I D3)**
Span ± (nm)	Half-width of each sweep
Step (nm)	Step size during the sweep
Dwell (s)	Per-point time (uses the current averaging settings)

Press **Calibrate**, watch both traces appear, then **Apply & Save** to write the measured value into the Slip field and persist it.

Peak position is taken as the **intensity-weighted centroid** of all points above 30 % of the peak height — sub-step accurate and far less noise sensitive than a plain maximum.

---

6. Moving: Go To, Jog, Mark Point

- **Go To** — absolute move to the entered wavelength.
- **Jog − / Jog +** — relative move by *Jog step*.
- **Mark Point** — takes one averaged reading at the current position and adds it to the plot as a marker. Useful for spot checks without a scan.

---

7. Scanning

Stepped scan (*Start Scan*)

Moves to Start, then steps to End in increments of *Step*, taking one averaged measurement at every point. This is the accurate mode: each point has a defined wavelength and a defined averaging window.

Targets are computed from an **absolute grid** anchored once at the start of the scan, so a step that falls slightly short is corrected by the next move instead of accumulating over the scan.

Free Run (*Free Run Scan (continuous)*)

One continuous move from Start to End while polling position and signal as fast as the link allows (~19–20 Hz on this rig). Fast, but:

- *Step* and *Timebase* do not apply — there is no per-point loop.
- No averaging, no Pause.
- Point spacing depends on speed, so it is uneven.
- Speed is capped at `RAMP_SP`; the field can only make it **slower**.

Free Run is for finding a line quickly. Use a stepped scan to measure it.

Pause / Resume / Stop

Pause suspends between points. **Stop** aborts and brings the drive to a safe halt — a single press is enough.

Swap Start / End

The ⇅ button between the labels and the fields swaps Start and End, so the next scan runs the opposite direction.

---

8. Averaging: Timebase, Pre-settle, Avg mode

This is the part most worth understanding, because it determines the noise floor of every measurement.

Timebase is the *total time budget for one scan point*. Everything else comes out of it:

```
|<-- pre-settle -->|<-- averaging window -->|<-- idle remainder -->|
|<------------------------ Timebase ----------------------------->|
```

Field	Meaning
Timebase (ms)	Total budget per point. (Formerly "Wait / dwell")
Pre-settle (ms)	Dead time after arriving, before measuring, so mechanics and signal settle. Applies in **both** averaging modes.
Avg mode	`samples` or `time`
Avg samples	`samples` mode: fixed number of DAQ reads
Avg fraction (%)	`time` mode: percentage of the time remaining **after** pre-settle

`samples` mode averages a fixed number of reads, independent of the clock → a constant noise floor per point regardless of Timebase. Any remaining time is idle.

`time` mode averages continuously for *Avg fraction* % of `Timebase − Pre-settle`. So 50 % always means half of the usable measurement time, whatever the pre-settle is.

The grey line under the fields translates the settings into milliseconds live, and after the first measured point it also reports the **measured** read rate:

```
1000 ms timebase = 200 ms pre-settle + 800 ms available.
50 % of that = 400 ms averaging, 400 ms idle.
Last point: 402 ms, 8 reads (19.9 Hz).
```

> **Compatibility note.** Before 2026-07-23 the fraction was taken from the *full* Timebase and the remaining time only acted as a cap, so with 1000 ms / 200 ms every setting from 80 % upward produced the same 800 ms window. `time`-mode scans recorded before and after that change are not directly comparable in noise floor.

---

9. The scan queue

Set up a scan in the fields, press **Add from fields**. The entry stores its **own complete acquisition settings**, not just the wavelength range:

```
01: 589.000 → 588.000, Δ 0.100 nm  |  100ms tb, 20ms pre, avg 8 samples
02: 589.000 → 588.000, Δ 0.100 nm  |  500ms tb, 20ms pre, avg time 60%
```

Button	Effect
Add from fields	Append a new entry with the current settings
Update selected	Overwrite the selected entry with the current settings
Remove selected	Delete the selected entry
Clear queue	Empty the queue
Run queue	Run every entry in order

Because each entry carries its own settings, changing a field after queueing does **not** affect entries already in the queue — select the entry and press *Update selected*.

After a queue finishes you are offered a bulk export of all scans plus a summary CSV.

---

10. The plot

- **Legend** — one entry per scan, in the curve's colour, deliberately short and in the same format for both scan types:

Scan type	Legend entry
Stepped	`22:06:24 589→588 Δ0.1 x8`
Free Run	`22:06:31 589→588 free 10000rpm`

Both start with `HH:MM:SS`. A stepped scan shows its step and a compact averaging token (`x8` = 8 samples, `60%` = time mode at 60 %); a free run has neither, so it shows its sweep speed instead — the one setting that distinguishes two otherwise identical free runs. The legend sits on top of the data, so it only has to tell curves apart — the full acquisition settings are in the log and in the CSV header. Its text size is set by `legend_font_px` in `vrs41_settings.json` and defaults to 13 px, the same size as the axis labels; raise it for a high-DPI display. The change takes effect on restart. - **Crosshair** — follows the mouse; the header shows *Cursor λ* and *Cursor V*. It survives *Clear Plot*. - **Traces ▾** — show, hide or delete an **individual** curve. Hiding keeps the data (it can be brought back with *Show all*); *Remove this trace* discards it and its legend entry. Saved CSVs are never affected. - **Save PNG** — writes the plot as an image. The export uses a **white** background and 1600 px width, because the dark on-screen theme is for the screen, not for a report. For full control over size, background and format (SVG, CSV of the plotted data) use the plot's right-click menu → *Export*. - **Clear Plot** — removes **all** traces at once. Saved CSVs are untouched.

There is **no light/dark mode** — the interface is dark only. The one place that matters for output is the PNG export, which already switches to a light background for you. - **Zoom / pan** — mouse wheel and drag (pyqtgraph). Right-click for the standard menu including *Export*.

---

11. The live AI monitor

The status bar continuously shows the analog input so the PMT / high voltage can be adjusted **without starting a scan**: numeric value, a colour-coded bar, a max-hold value and a *Reset max* button.

Range	Colour
below `AI_WARN_V` (6 V)	green
`AI_WARN_V` … `AI_ALARM_V` (6–7 V)	yellow
above `AI_ALARM_V` (7 V)	red

Polled at 4 Hz **while idle only**. During a scan it displays the value the scan itself just measured — opening a second NI-DAQ task on a channel the scan thread has reserved would fail, so the monitor never competes for it.

If the label reads `AI (SIM)`, the values are simulated (§18).

---

12. Export and file formats

When is a scan written to disk?

Event	With Autosave on	With Autosave off
Scan finishes normally	Written automatically, `status: completed`	Not written
Stop pressed	**Written automatically**, `status: interrupted` — the points measured so far are kept	Not written
Queue finishes	Every scan written, plus an optional bulk export and summary	Bulk export offered
Save CSV now pressed	Writes the active scan immediately, at any time	Same

So a stopped scan is **not** lost: with Autosave on, the partial data is saved and marked `interrupted` in the header, so it can never be mistaken for a complete measurement. To keep a scan that is still running, press *Save CSV now* — it exports what has been measured up to that moment and can be pressed repeatedly.

Folder and filename pattern are set in the EXPORT section.

The header carries the full acquisition context:

```
# scan_id: ...
# started_at / finished_at / status
# start_nm / end_nm / step_nm / wait_s
# avg_mode / avg_samples / avg_fraction_pct / pre_settle_ms
# signal_source: NI-DAQ
# columns: index,wavelength_nm,voltage_V
index,wavelength_nm,voltage_V
0,589.000000,0.070880
```

`signal_source` is either `NI-DAQ` or `SIMULATED`; simulated files carry an extra explicit warning line. **Check this line before trusting a file** — the simulator produces realistic-looking spectra.

---

13. Reading the log

The Console tab shows everything. Prefixes:

Prefix	Meaning
`[OK]`	Normal progress
`[FSM]`	State change: `IDLE`, `MOVING`, `SCAN`
`[REF]`	Reference run
`[CAL]`	Wavelength re-anchoring
`[SLIP]`	Backlash compensation applied on a reversal
`[OST]` / `[AUDIT]`	Drive operation status after a move
`[FREE RUN]`	Free-run progress, speed, re-anchoring
`[CSV]`	File written
`[DONE]`	Scan or move finished
`[STOP]`	Aborted by the user
`[TIMEOUT]`	A move did not reach its target in time
`[SERIAL]`	Incomplete/garbled reply, discarded
`[RESYNC]`	No complete reply; buffer drained
`[DAQ]`	DAQ driver availability
`[WARN]` / `[SERIAL ERROR]`	Problems

`[OST] 0x0500 (IN1|IN3)` is the *normal* idle state on this rig: both limit-switch inputs are at their rest level. The switches are **active low** — a pressed endstop *clears* its bit (§19).

---

14. When something goes wrong

Scan will not start, "start/end/step must be numbers"

Check the Step field. Both `0.1` and `0.10` are valid; if it looks fine, retype the value.

`[TIMEOUT] Did not reach target`

The move did not arrive in the allotted time. Causes: the drive is blocked (endstop, mechanical obstruction), the speed was reduced so far that the budget ran out, or the tolerance is tighter than the servo can hold. Check the drive and the Free Run speed setting.

`[SERIAL] Incomplete reply` / **`[RESYNC]`**

A reply arrived truncated or not at all. Occasional lines are harmless — the sample is skipped, never guessed. A continuous stream of them means the serial link is degraded: check cable routing away from the motor leads, grounding, and the USB-serial adapter. `idle_poll_test.py` (§17) separates a link problem from a motion-correlated one in two minutes.

The wavelength is obviously wrong

Re-anchor with *Known λ here* (§4). If a second line is then also off, it is a scale error, not an offset.

Repeated scans drift

Run the same scan several times and compare peak positions. Drift that scales with the **number of steps** points at positioning; drift that scales with the number of **reversals** points at backlash. See BACKLOG.md for the measurements behind this.

The drive is stuck after a fault

Press **Stop** — it stops the move, then re-arms the drive (`ST`, `EN`, `V0`) once the running worker has actually exited.

---

15. Settings: what is saved where

Two separate files, with a clear split:

File	Contents	Edited by
`vrs41_settings.json`	Runtime settings that change per session	The GUI, automatically
`device_constants.py`	Hardware and timing constants	A developer, in the editor

`vrs41_settings.json`

Written next to the program. All keys are defined in `app_config.DEFAULT_CONFIG`; a missing or corrupt file falls back to defaults rather than crashing.

Key	Meaning
`backlash_slip_nm`	Measured backlash
`last_current_nm`	Last known wavelength (software estimate, **not** a readback)
`port`, `baud`	Connection
`scan_start_nm`, `scan_end_nm`, `scan_step_nm`, `scan_wait_ms`	Scan fields
`presettle_ms`, `avg_mode`, `avg_samples`, `avg_fraction_pct`	Averaging
`free_run_sp_rpm`	Free Run speed
`jog_step_nm`	Jog step
`export_dir`, `export_pattern`, `autosave_csv`	Export
`legend_font_px`	Plot legend text size in pixels (default 13 = same as the axis labels). Takes effect on restart
`daq_dev`, `daq_ai`	DAQ device and channel
`cal_center_nm`, `cal_span_nm`, `cal_step_nm`, `cal_dwell_s`	Calibration dialog

Fields save themselves when you leave them (focus change or Enter), **and** everything is saved again when the window closes — so a value typed just before quitting is not lost.

> Two separate bugs used to defeat this and were fixed on 2026-07-23. Saving only happened on `editingFinished`, so a value typed immediately before closing was lost; and on startup the

> entry *adapters* were constructed with hardcoded defaults that overwrote the values just loaded from the JSON — so `avg_samples` and `avg_fraction_pct` came back as 8 and 100 every session no matter what the file said.

Deliberately not saved: the backlash seating direction. It describes the physical state of the gear train, which a closed program cannot know, so the Reference Run stays a per-session action.

---

16. Constants reference

All in `device_constants.py`, grouped by topic. The ones you are most likely to touch:

Mechanics and geometry

Constant	Value	Meaning
`STEPS_PER_NM`	361765	Encoder steps per nm (≈120.6 motor turns/nm)
`INVERT_DIRECTION`	True	Sign convention of this rig

Motion / ramp

Constant	Meaning
`RAMP_AC`, `RAMP_DEC`, `RAMP_SP`	Acceleration, deceleration, speed. `RAMP_SP` is the ceiling for Free Run
`SEC_PER_NM`, `TIMEOUT_OVERHEAD_S`, `MIN/MAX_MOVE_TIMEOUT`	Move timeout budget
`POS_TOL_STEPS`, `POS_STABLE_COUNT`	Arrival tolerance and how many consecutive readings confirm it
`SAFE_AFTER_MOVE`	Park the drive after each move
`STOP_JOIN_TIMEOUT_S`	How long Stop waits for a worker to exit before re-arming

Serial protocol

Constant	Meaning
`MOTOR_ANSW_MODE`	Drive answer mode (0 = silent)
`NO_REPLY_COMMANDS`	Commands sent fire-and-forget in silent mode
`POLL_TIMEOUT`	Total wait for one reply
`REPLY_GAP_TOLERANCE_S`	Longest tolerated pause *between bytes* of one reply
`REQUIRE_CR_TERMINATED_REPLY`	Reject replies that never finished
`SERIAL_RECOVERY_SETTLE_S`, `WRITE_SETTLE_S`, `RESYNC_ON_TIMEOUT`	Recovery behaviour
`SEND_LEGACY_HP0`	Legacy `HP0` write (off; `HP` is limit-switch polarity, not a motion command)

DAQ and display

Constant	Meaning
`DAQ_SAMPLES_PER_READ`, `DAQ_SAMPLE_RATE_HZ`	Hardware samples per read and rate
`AI_WARN_V`, `AI_ALARM_V`, `AI_BAR_MAX_V`	Monitor thresholds and full scale
`AI_MONITOR_HZ`, `AI_MONITOR_ENABLED`	Monitor refresh
`OST_ENDSTOP_ACTIVE_LOW`	Endstop polarity (measured: active low)

UI theme

All interface colours live in one block in `device_constants.py` (section *UI THEME*) so the look can be adjusted by hand without touching `gui_main.py`. Changes take effect on restart.

Constant	Meaning
`C_BG`, `C_PANEL`, `C_PANEL2`, `C_BORDER`	Surfaces and borders
`C_TEXT`, `C_MUTED`	Normal and secondary text
`C_ACCENT`, `C_ACCENT_D`, `C_ON_ACCENT`	Teal accent, its pressed state, and text on it
`C_WARN`, `C_ERROR`, `C_OK`, `C_AMBER`	State colours (`C_AMBER` is the AI monitor's warn band)
`C_DISABLED_FG`, `C_DISABLED_BD`	Disabled buttons
`C_LEGEND_BG`	Legend backdrop, as `(R, G, B, A)`
`PLOT_PALETTE`	**16** trace colours, used in order and then cycled

The palette was picked numerically, not by eye: every colour has at least 5.1:1 contrast against the background, the closest pair is 26.6 CIELAB ΔE apart, and consecutive entries are at least 53 ΔE apart so two scans run back to back never look alike. If you edit it, keep those three properties in mind.

Defaults and simulator

Constant	Meaning
`AVG_MODE_DEFAULT`, `AVG_SAMPLES_DEFAULT`, `AVG_FRACTION_PCT_DEFAULT`, `PRESETTLE_MS_DEFAULT`	Averaging defaults
`CAL_CENTER_NM_DEFAULT`	Reference line — **587.5618 nm (He I D3)**
`SIM_BACKLASH_NM`	Backlash modelled by the simulator

---

17. Diagnostic scripts

Standalone command-line tools. All are read-only with respect to the drive's EEPROM — none of them ever writes a permanent setting.

`read_faulhaber_config.py`

Dumps and **decodes** the drive's entire configuration: identity, answer mode, operating mode, encoder resolution, limit-switch wiring (`IOC`), homing config, position-loop tuning, and a live snapshot including housing temperature.

```
python read_faulhaber_config.py --baud 9600 COM3
```

Use it when the drive behaves in a way the software cannot explain, or after anyone has been in Motion Manager. It warns explicitly if both hard-blocking inputs lock the same direction of travel.

`idle_poll_test.py`

Polls position at ~20 Hz for two minutes **with the motor completely idle**, counting truncated or failed replies.

```
python idle_poll_test.py COM3 --duration 120 --csv idle.csv
```

This is the decisive test when serial errors appear during scans: no errors at rest but errors while moving means the cause is motion-correlated (EMI, grounding, cable routing); errors at rest too means the link or driver itself.

`step_resolution_test.py`

Determines how small a scan step this rig can actually resolve and how tight `POS_TOL_STEPS` can safely be.

```
python step_resolution_test.py COM3 --steps 0.01 0.001 0.0001 --moves 20
```

For each step size it records the settling curve and the **final** resting error, then evaluates candidate tolerances using the same rule the real move loop uses (consecutive in-band readings, no credit for briefly passing through). It prints the tightest tolerance that worked on every move.

`probe_move_timing.py`

Measures real move and poll timing on this rig — where the ~19–20 Hz Free Run rate comes from, and how long moves of a given size actually take.

`tune_ramp.py`

Experiments with acceleration / deceleration / speed settings.

---

18. Simulator mode

Enter **`SIM`** as the port. A virtual drive replaces the serial hardware and, if no NI-DAQ driver is present, a synthetic spectrum replaces the PMT.

The simulator is deliberately realistic:

- **Mechanical backlash** (`SIM_BACKLASH_NM`, default 0.082 nm). Encoder and optics are tracked separately: on a direction reversal the first `SIM_BACKLASH_NM` of travel is absorbed by slack — the encoder counts it, the grating does not move. The simulated spectrum follows the *optics*.
- **Emission lines** at 486.1 (H-β), 587.5618 (He I D3), 589.3 (Na-D) and 656.3 nm (H-α), on a baseline, with noise.
- **Silent answer mode**, matching the real drive, including `CR` + `LF` reply termination.

This makes backlash compensation, the calibration routine, scan drift and the whole export path testable without hardware.

> **Simulated data is always labelled.** The footer shows `AI (SIM)`, a warning is logged at startup, and every CSV carries `# signal_source: SIMULATED`. Check that line before trusting a file — the synthetic spectrum looks like a real one, at plausible wavelengths.

---

19. Known limits

Honest list. Details and measurements are in `BACKLOG.md`.

- **No absolute position sensor.** Everything rests on the anchor you provide. Restart, hand movement or a missed step invalidates it.
- **Endstops are handled by the drive, not by this software.** Inputs 1 and 3 are configured as hard-blocking limit switches in the controller, which physically blocks motion. The application's own endstop guard is dormant and its logic is inverted relative to the measured active-low polarity, so it is deliberately left disabled rather than enabled untested. The app notices a blocked drive only indirectly, via a deviation fault.
- **Both limit switches block the same direction of travel** (`HD = 00101`). For a pair of end-of-travel switches this means only one end is actually protected. Changing it requires writing to the drive and has not been done.
- **`POS_TOL_STEPS` does not scale with the step size.** It is a fixed 0.01 nm, which equals a whole step at 0.01 nm and is ten times the step at 0.001 nm. The absolute grid stops errors accumulating, but individual fine steps may still sit off their nominal position. Use `step_resolution_test.py` before scanning finer than 0.01 nm.
- **Free Run point spacing is uneven** and depends on speed and link performance.
- **`LR` is relative to the last commanded setpoint, not the actual position** (per the drive manual). After a hard stop the drive's internal reference and `POS` can differ.

Document generated from `MANUAL.md` — the single source. The PDF and the GitHub README are both produced from this file.